==========

C++ PRACTICE QUESTIONS

==========

1. Foundations & Compilation

1. Write a program that prints "Hello C++" and your name on two different lines
   WITHOUT using two separate cout statements.

2. Write a program that takes two integers as input and prints their sum
   ONLY if both numbers are positive; otherwise print "Invalid input".

3. Declare variables of type int, float, char, and bool.
   Print their sizes using sizeof and explain why the size of bool is not 1 bit.

4. Write a program to swap two numbers using a third variable.
   Print the values before and after swapping to verify correctness.

5. Create a constant variable for PI and calculate the area of a circle.
   If the radius entered is negative, display "Invalid radius".

6. Write a program that uses cin to take the user's age.
   - If age >= 100, print "Already 100 or more"
   - Otherwise, print how many years are left to reach 100.

7. Implement a simple calculator using a switch statement for +, -, *, and /.
   Handle division by zero gracefully.

8. Write a for loop that prints all even numbers from 1 to 20,
   but SKIP numbers that are divisible by 4.

9. Use a while loop to calculate the factorial of a number entered by the user.
   If the user enters a negative number, keep asking until a valid number is entered.

10. Write a program that checks whether a number is positive, negative, or zero.
    If the number is non-zero, also check whether it is even or odd.

---

2. Data Structures & Memory Management

---

1. Create an array of 5 integers.
   Calculate and print both the sum and average of its elements.

2. Write a program to find the largest number in an array of 10 elements
   and also print its index position.

3. Declare an integer and a pointer to that integer.
   Modify the value of the integer ONLY using the pointer.

4. Create a struct called Student with name and roll number.
   Pass the structure to a function and display its data.

5. Dynamically allocate memory for a single integer using new.
   Check whether memory allocation was successful before using it.

6. Write a program to reverse a string entered by the user
   WITHOUT using built-in reverse functions.

7. Create an array and access all its elements using pointer arithmetic
   (do not use array indexing).

8. Dynamically allocate an array of size n entered by the user,
   store values in it, display them, and free the memory using delete[].

9. Create a struct Rectangle with length and width.
   Write a function that takes the structure and returns the area.

10. Write a program that uses reference variables to swap two integers
    WITHOUT using a third variable.

---

3. Object-Oriented Programming (OOP)

---

1. Create a class Car with attributes brand and year.
   Add a method that prints "Old Car" if year < 2015, else "New Car".

2. Create a Calculator class with an add() function.
   Overload add() to work with both integers and floating-point values.

3. Create a class with private data members.
   Use setter functions that validate input before assigning values.

4. Create a Book class with a constructor that initializes title and author.
   Use a static variable to count how many Book objects are created.

5. Create a Bottle class with a destructor that prints "Bottle destroyed".
   Demonstrate destructor calls using function scope.

6. Use the this pointer in a class constructor to resolve naming conflicts
   between member variables and parameters.

7. Create a class with a static variable that tracks how many objects
   currently exist (increment in constructor, decrement in destructor).

8. Create a Box class with private dimensions.
   Write a friend function that compares two boxes and prints the larger one.

9. Create an array of Employee objects.
   Display only those employees whose salary is above a given threshold.

10. Create a Point class with X and Y coordinates.
    Add a function to calculate the distance of the point from the origin.

---

4. Inheritance & Polymorphism

---

1. Create a base class Animal with a function type().
   Override it in a derived class Dog.

2. Demonstrate method overriding by calling functions using a base class pointer.

3. Implement multilevel inheritance:
   Person -> Employee -> Manager
   Show the order of constructor calls.

4. Demonstrate multiple inheritance and resolve ambiguity
   using the scope resolution operator.

5. Overload a function area() for calculating the area of a circle
   and a rectangle, based on user choice.

6. Overload the + operator to add two Box objects correctly.

7. Create a virtual function sound() in a base class.
   Show the difference between virtual and non-virtual behavior.

8. Create an abstract class Shape with a pure virtual function draw().
   Derive Circle and Rectangle classes.

9. Use a base class pointer to store addresses of different derived class objects and call overridden methods.

10. Create a Bird class with a virtual destructor.
Demonstrate why it is required using dynamic memory allocation.